Vishal Acharya

1. What is groupby() in Pandas?

groupby() is used to split data into groups, apply operations, and combine the results.

- It follows the Split → Apply → Combine paradigm:

1. Split the data into groups
2. Apply a function (sum, mean, count, custom logic)
3. Combine the results into a new object

df.groupby(by, axis=0, level=None, as_index=True, sort=True)

Parameter	Meaning
`by`	Column name(s), list of columns, function, or mapping
`axis`	`0` → group rows (default), `1` → group columns
`as_index`	`True` → group keys become index
`sort`	`True` → sort group keys
`level`	Group by index level (for MultiIndex)

Internal Working (Very Important Concept)

- Think of groupby() as:
- Split → Apply → Combine

Step	Explanation
Split	Data divided into groups

Vishal Acharya

Step	Explanation
Apply	Function applied to each group
Combine	Results merged

Task	Method
Group data	`groupby()`
Aggregate	`sum()`, `mean()`, `count()`
Multiple stats	`agg()`
Row-wise result	`transform()`
Custom logic	`apply()`
Access group	`get_group()`

by Parameter – Grouping Column(s)

Group by one column

In [1]:
```
import pandas as pd

data = {
    "Department": ["IT", "IT", "HR", "HR", "Finance", "IT"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
    "Salary": [50000, 60000, 45000, 48000, 70000, 55000],
    "Experience": [2, 3, 1, 2, 5, 4]
}

df = pd.DataFrame(data)
print(df)

df.groupby("Department").mean()
```

Vishal Acharya

```
  Department Employee  Salary  Experience
0         IT        A   50000           2
1         IT        B   60000           3
2         HR        C   45000           1
3         HR        D   48000           2
4    Finance        E   70000           5
5         IT        F   55000           4
```

Out[1]:

Department	Salary	Experience
Finance	70000.0	5.0
HR	46500.0	1.5
IT	55000.0	3.0

Group by multiple columns

In [2]:

```python
import pandas as pd

data = {
    "Department": ["IT", "IT", "HR", "HR", "Finance", "IT"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
    "Salary": [50000, 60000, 45000, 48000, 70000, 55000],
    "Experience": [2, 3, 1, 2, 5, 4]
}

df = pd.DataFrame(data)
print(df)

df.groupby(["Department", "Experience"]).sum()
```

```
  Department Employee  Salary  Experience
0         IT        A   50000           2
1         IT        B   60000           3
2         HR        C   45000           1
3         HR        D   48000           2
4    Finance        E   70000           5
5         IT        F   55000           4
```

Vishal Acharya

Out[2]:

Department	Experience	Salary
Finance	**5**	70000
HR	**1**	45000
	2	48000
IT	**2**	50000
	3	60000
	4	55000

axis Parameter

- Default (axis=0) → group rows
- axis=1 → group columns (rare but useful)

COLUMNS

↓

Apply lambda to column names

↓

Create column groups

↓

Apply sum() row-wise

Vishal Acharya

↓

Return combined DataFrame

In [7]:
```
import pandas as pd

data = {
    "Department": ["IT", "IT", "HR", "HR", "Finance", "IT"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
    "Salary": [50000, 60000, 45000, 48000, 70000, 55000],
    "Experience": [2, 3, 1, 2, 5, 4]
}

df = pd.DataFrame(data)
print(df)
df.groupby(lambda x: "Pay" if "Salary" in x else "Other", axis=1).sum()
```

```
  Department Employee  Salary  Experience
0         IT        A   50000           2
1         IT        B   60000           3
2         HR        C   45000           1
3         HR        D   48000           2
4    Finance        E   70000           5
5         IT        F   55000           4
```

Out[7]:

	Other	Pay
0	2	50000
1	3	60000
2	1	45000
3	2	48000
4	5	70000
5	4	55000

Vishal Acharya

as_index Parameter

Default: as_index=True

In [1]:
```
import pandas as pd

data = {
    "Department": ["IT", "IT", "HR", "HR", "Finance", "IT"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
    "Salary": [50000, 60000, 45000, 48000, 70000, 55000],
    "Experience": [2, 3, 1, 2, 5, 4]
}

df = pd.DataFrame(data)
print(df)
df.groupby("Department").sum()
```

```
  Department Employee  Salary  Experience
0         IT        A   50000           2
1         IT        B   60000           3
2         HR        C   45000           1
3         HR        D   48000           2
4    Finance        E   70000           5
5         IT        F   55000           4
```

Out[1]:

Department	Salary	Experience
Finance	70000	5
HR	93000	3
IT	165000	9

In [2]:
```
df.groupby("Department", as_index=False).sum()
```

Vishal Acharya

Out[2]:

	Department	Salary	Experience
0	Finance	70000	5
1	HR	93000	3
2	IT	165000	9

Sort Parameter

▶️ Default: sort=True

In [3]:

```
import pandas as pd

data = {
    "Department": ["IT", "IT", "HR", "HR", "Finance", "IT"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
    "Salary": [50000, 60000, 45000, 48000, 70000, 55000],
    "Experience": [2, 3, 1, 2, 5, 4]
}

df = pd.DataFrame(data)
print(df)
df.groupby("Department", sort=True).sum()
```

```
  Department Employee  Salary  Experience
0         IT        A   50000           2
1         IT        B   60000           3
2         HR        C   45000           1
3         HR        D   48000           2
4    Finance        E   70000           5
5         IT        F   55000           4
```

Vishal Acharya

Out[3]:

Department	Salary	Experience
Finance	70000	5
HR	93000	3
IT	165000	9

In [4]:
```
df.groupby("Department", sort=False).sum()
```

Out[4]:

Department	Salary	Experience
IT	165000	9
HR	93000	3
Finance	70000	5

Aggregation Functions with groupby()

▶️ Single aggregation

In [5]:
```
import pandas as pd

data = {
    "Department": ["IT", "IT", "HR", "HR", "Finance", "IT"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
    "Salary": [50000, 60000, 45000, 48000, 70000, 55000],
    "Experience": [2, 3, 1, 2, 5, 4]
}

df = pd.DataFrame(data)
print(df)
df.groupby("Department")["Salary"].sum()
```

Vishal Acharya

```
  Department Employee  Salary  Experience
0         IT        A   50000           2
1         IT        B   60000           3
2         HR        C   45000           1
3         HR        D   48000           2
4    Finance        E   70000           5
5         IT        F   55000           4
```

Out[5]:

```
Department
Finance     70000
HR          93000
IT         165000
Name: Salary, dtype: int64
```

Multiple aggregations

In [6]:

```python
import pandas as pd

data = {
    "Department": ["IT", "IT", "HR", "HR", "Finance", "IT"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
    "Salary": [50000, 60000, 45000, 48000, 70000, 55000],
    "Experience": [2, 3, 1, 2, 5, 4]
}

df = pd.DataFrame(data)
print(df)
df.groupby("Department").agg({
    "Salary": ["min", "max", "mean"],
    "Experience": "mean"
})
```

```
  Department Employee  Salary  Experience
0         IT        A   50000           2
1         IT        B   60000           3
2         HR        C   45000           1
3         HR        D   48000           2
4    Finance        E   70000           5
5         IT        F   55000           4
```

Vishal Acharya

Out[6]:

	Salary			Experience
	min	max	mean	mean
Department				
Finance	70000	70000	70000	5.0
HR	45000	48000	46500	1.5
IT	50000	60000	55000	3.0

Accessing Individual Groups

Returns only IT department rows

In [7]:

```
import pandas as pd

data = {
    "Department": ["IT", "IT", "HR", "HR", "Finance", "IT"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
    "Salary": [50000, 60000, 45000, 48000, 70000, 55000],
    "Experience": [2, 3, 1, 2, 5, 4]
}

df = pd.DataFrame(data)
print(df)
grouped = df.groupby("Department")
grouped.get_group("IT")
```

```
  Department Employee  Salary  Experience
0         IT        A   50000           2
1         IT        B   60000           3
2         HR        C   45000           1
3         HR        D   48000           2
4    Finance        E   70000           5
5         IT        F   55000           4
```

Vishal Acharya

Out[7]:

	Department	Employee	Salary	Experience
0	IT	A	50000	2
1	IT	B	60000	3
5	IT	F	55000	4

Groupby with Custom Function

Computes salary range per department

In [9]:

```
import pandas as pd

data = {
    "Department": ["IT", "IT", "HR", "HR", "Finance", "IT"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
    "Salary": [50000, 60000, 45000, 48000, 70000, 55000],
    "Experience": [2, 3, 1, 2, 5, 4]
}

df = pd.DataFrame(data)
print(df)
df.groupby("Department")["Salary"].apply(lambda x: x.max() - x.min())
```

```
  Department Employee  Salary  Experience
0         IT        A   50000           2
1         IT        B   60000           3
2         HR        C   45000           1
3         HR        D   48000           2
4    Finance        E   70000           5
5         IT        F   55000           4
```

Out[9]:

```
Department
Finance        0
HR          3000
IT         10000
Name: Salary, dtype: int64
```

Groupby with transform() (Advanced)

Vishal Acharya

Keeps same shape as original DataFrame

In [10]:
```
import pandas as pd

data = {
    "Department": ["IT", "IT", "HR", "HR", "Finance", "IT"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
    "Salary": [50000, 60000, 45000, 48000, 70000, 55000],
    "Experience": [2, 3, 1, 2, 5, 4]
}

df = pd.DataFrame(data)
print(df)
df["Dept_Avg_Salary"] = df.groupby("Department")["Salary"].transform("mean")
print(df)
```

```
  Department Employee  Salary  Experience
0         IT        A   50000           2
1         IT        B   60000           3
2         HR        C   45000           1
3         HR        D   48000           2
4    Finance        E   70000           5
5         IT        F   55000           4
  Department Employee  Salary  Experience  Dept_Avg_Salary
0         IT        A   50000           2            55000
1         IT        B   60000           3            55000
2         HR        C   45000           1            46500
3         HR        D   48000           2            46500
4    Finance        E   70000           5            70000
5         IT        F   55000           4            55000
```

groupby() + nth()

- What is nth()?

nth() is used with groupby() to select the nth row from each group.

Think of it as: “Give me the nth record from every group

Vishal Acharya

df.groupby("column").nth(n)

n can be:

positive → from start

negative → from end

list → multiple positions

Real-World Uses of nth()

1. First student per class
2. Latest transaction per account
3. First login per user
4. Second attempt marks

First row from each group (nth(0))

First occurrence per department

In [11]:
```
import pandas as pd

data = {
    "Department": ["IT", "IT", "IT", "HR", "HR", "Finance"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
    "Salary": [50000, 60000, 55000, 45000, 48000, 70000]
}

df = pd.DataFrame(data)
print(df)
df.groupby("Department").nth(0)
```

Vishal Acharya

```
  Department Employee  Salary
0         IT        A   50000
1         IT        B   60000
2         IT        C   55000
3         HR        D   45000
4         HR        E   48000
5    Finance        F   70000
```

Out[11]:

Department	Employee	Salary
Finance	F	70000
HR	D	45000
IT	A	50000

Second row from each group (nth(1))

Finance missing → only one row exists

In [12]:
```
df.groupby("Department").nth(1)
```

Out[12]:

Department	Employee	Salary
HR	E	48000
IT	B	60000

Last row from each group (nth(-1))

Very common in latest record analysis

In [13]:
```
import pandas as pd

data = {
    "Department": ["IT", "IT", "IT", "HR", "HR", "Finance"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
```

Vishal Acharya

```
    "Salary": [50000, 60000, 55000, 45000, 48000, 70000]
}

df = pd.DataFrame(data)
print(df)
df.groupby("Department").nth(-1)
```

```
  Department Employee  Salary
0         IT        A   50000
1         IT        B   60000
2         IT        C   55000
3         HR        D   45000
4         HR        E   48000
5    Finance        F   70000
```

Out[13]:

Department	Employee	Salary
Finance	F	70000
HR	E	48000
IT	C	55000

Multiple positions

First and last record of each group

In [14]:

```
import pandas as pd

data = {
    "Department": ["IT", "IT", "IT", "HR", "HR", "Finance"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
    "Salary": [50000, 60000, 55000, 45000, 48000, 70000]
}

df = pd.DataFrame(data)
print(df)
df.groupby("Department").nth([0, -1])
```

Vishal Acharya

```
  Department Employee  Salary
0         IT        A   50000
1         IT        B   60000
2         IT        C   55000
3         HR        D   45000
4         HR        E   48000
5    Finance        F   70000
```

Out[14]:

Department	Employee	Salary
Finance	F	70000
HR	D	45000
HR	E	48000
IT	A	50000
IT	C	55000

Keep missing groups (dropna=False)

Keeps rows even if nth index doesn't exist

In [15]:

```python
import pandas as pd

data = {
    "Department": ["IT", "IT", "IT", "HR", "HR", "Finance"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
    "Salary": [50000, 60000, 55000, 45000, 48000, 70000]
}

df = pd.DataFrame(data)
print(df)
df.groupby("Department").nth(1, dropna=False)
```

Vishal Acharya

```
  Department Employee  Salary
0         IT        A   50000
1         IT        B   60000
2         IT        C   55000
3         HR        D   45000
4         HR        E   48000
5    Finance        F   70000
```

Out[15]:

Department	Employee	Salary
HR	E	48000
IT	B	60000

In [17]:
```python
df.groupby("Department").nth(1, dropna=True)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-17-919938a9f250> in <module>
----> 1 df.groupby("Department").nth(1, dropna=True)

C:\ProgramData\Anaconda3\lib\site-packages\pandas\core\groupby\groupby.py in nth(self, n, dropna)
   1961         if dropna not in ["any", "all"]:
   1962             # Note: when agg-ing picker doesn't raise this, just returns NaN
-> 1963             raise ValueError(
   1964                 "For a DataFrame groupby, dropna must be "
   1965                 "either None, 'any' or 'all', "

ValueError: For a DataFrame groupby, dropna must be either None, 'any' or 'all', (was passed True).
```

In [24]:
```python
def label_high_salary(group):
    group["High_Earner"] = group["Salary"] > group["Salary"].mean()
    return group
import pandas as pd

data = {
    "Department": ["IT", "IT", "IT", "HR", "HR", "Finance"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
    "Salary": [50000, 60000, 55000, 45000, 48000, 70000]
```

Vishal Acharya

```
}

df = pd.DataFrame(data)
print(df)
df_new = df.groupby("Department").apply(label_high_salary)
print(df_new)
```

```
  Department Employee  Salary
0         IT        A   50000
1         IT        B   60000
2         IT        C   55000
3         HR        D   45000
4         HR        E   48000
5    Finance        F   70000
  Department Employee  Salary  High_Earner
0         IT        A   50000        False
1         IT        B   60000         True
2         IT        C   55000        False
3         HR        D   45000        False
4         HR        E   48000         True
5    Finance        F   70000        False
```

TASK-1

In [28]:
```
import pandas as pd
import numpy as np
```

In [29]:
```
movies = pd.read_csv('imdb-top-1000.csv')
```

In [30]:
```
movies.head()
```

Vishal Acharya

Out[30]:

	Series_Title	Released_Year	Runtime	Genre	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
0	The Shawshank Redemption	1994	142	Drama	9.3	Frank Darabont	Tim Robbins	2343110	28341469.0	80.0
1	The Godfather	1972	175	Crime	9.2	Francis Ford Coppola	Marlon Brando	1620367	134966411.0	100.0
2	The Dark Knight	2008	152	Action	9.0	Christopher Nolan	Christian Bale	2303232	534858444.0	84.0
3	The Godfather: Part II	1974	202	Crime	9.0	Francis Ford Coppola	Al Pacino	1129952	57300000.0	90.0
4	12 Angry Men	1957	96	Crime	9.0	Sidney Lumet	Henry Fonda	689845	4360000.0	96.0

In [33]:

```
genres = movies.groupby('Genre')
genres
```

Out[33]:

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x0000028926FD6D30>
```

Applying builtin aggregation fuctions on groupby objects

In [34]:

```
genres.std()
```

Vishal Acharya

Out[34]:

Genre	Runtime	IMDB_Rating	No_of_Votes	Gross	Metascore
Action	28.500706	0.304258	432946.814748	2.256724e+08	12.421252
Adventure	33.317320	0.229781	301188.347642	1.697543e+08	12.345393
Animation	14.530471	0.253221	262173.231571	2.091840e+08	8.813646
Biography	25.514466	0.267140	271284.191372	1.363251e+08	11.028187
Comedy	22.946213	0.228771	188653.570564	1.946513e+08	11.829160
Crime	27.689231	0.335477	373999.730656	1.571191e+08	13.099102
Drama	27.740490	0.267229	305554.162841	2.201164e+08	12.744687
Family	10.606602	0.000000	137008.302816	3.048412e+08	16.970563
Fantasy	12.727922	0.141421	22179.111299	7.606861e+07	NaN
Film-Noir	4.000000	0.152753	54649.083277	7.048472e+07	1.527525
Horror	13.604812	0.311302	234883.508691	9.965017e+07	15.362291
Mystery	14.475423	0.310791	404621.915297	1.567524e+08	18.604435
Thriller	NaN	NaN	NaN	NaN	NaN
Western	17.153717	0.420317	263489.554280	1.230626e+07	9.032349

In [37]:

```
genres.min()
```

Vishal Acharya

Out[37]:

Genre	Series_Title	Released_Year	Runtime	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
Action	300	1924	45	7.6	Abhishek Chaubey	Aamir Khan	25312	3296.0	33.0
Adventure	2001: A Space Odyssey	1925	88	7.6	Akira Kurosawa	Aamir Khan	29999	61001.0	41.0
Animation	Akira	1940	71	7.6	Adam Elliot	Adrian Molina	25229	128985.0	61.0
Biography	12 Years a Slave	1928	93	7.6	Adam McKay	Adrien Brody	27254	21877.0	48.0
Comedy	(500) Days of Summer	1921	68	7.6	Alejandro G. Iñárritu	Aamir Khan	26337	1305.0	45.0
Crime	12 Angry Men	1931	80	7.6	Akira Kurosawa	Ajay Devgn	27712	6013.0	47.0
Drama	1917	1925	64	7.6	Aamir Khan	Abhay Deol	25088	3600.0	28.0
Family	E.T. the Extra-Terrestrial	1971	100	7.8	Mel Stuart	Gene Wilder	178731	4000000.0	67.0
Fantasy	Das Cabinet des Dr. Caligari	1920	76	7.9	F.W. Murnau	Max Schreck	57428	337574718.0	NaN
Film-Noir	Shadow of a Doubt	1941	100	7.8	Alfred Hitchcock	Humphrey Bogart	59556	449191.0	94.0
Horror	Alien	1933	71	7.6	Alejandro Amenábar	Anthony Perkins	27007	89029.0	46.0
Mystery	Dark City	1938	96	7.6	Alex Proyas	Bernard-Pierre Donnadieu	33982	1035953.0	52.0
Thriller	Wait Until Dark	1967	108	7.8	Terence Young	Audrey Hepburn	27733	17550741.0	81.0
Western	Il buono, il brutto, il cattivo	1965	132	7.8	Clint Eastwood	Clint Eastwood	65659	5321508.0	69.0

Vishal Acharya

In [40]:
```
genres.mean()
```

Out[40]:

Genre	Runtime	IMDB_Rating	No_of_Votes	Gross	Metascore
Action	129.046512	7.949419	420246.581395	1.897224e+08	73.419580
Adventure	134.111111	7.937500	313557.819444	1.319017e+08	78.437500
Animation	99.585366	7.930488	268032.073171	1.784326e+08	81.093333
Biography	136.022727	7.938636	272805.045455	9.404952e+07	76.240506
Comedy	112.129032	7.901290	178195.658065	1.010572e+08	78.720000
Crime	126.392523	8.016822	313398.271028	7.899656e+07	77.080460
Drama	124.737024	7.957439	212343.612457	1.225259e+08	79.701245
Family	107.500000	7.800000	275610.500000	2.195553e+08	79.000000
Fantasy	85.000000	8.000000	73111.000000	3.913633e+08	NaN
Film-Noir	104.000000	7.966667	122405.000000	4.197018e+07	95.666667
Horror	102.090909	7.909091	340232.363636	9.405902e+07	80.000000
Mystery	119.083333	7.975000	350250.333333	1.047014e+08	79.125000
Thriller	108.000000	7.800000	27733.000000	1.755074e+07	81.000000
Western	148.250000	8.350000	322416.250000	1.455538e+07	78.250000

find the top 3 genres by total earning

In [35]:
```
movies.groupby('Genre').sum()['Gross'].sort_values(ascending=False).head(3)
```

Vishal Acharya

Out[35]:
```
Genre
Drama     3.540997e+10
Action    3.263226e+10
Comedy    1.566387e+10
Name: Gross, dtype: float64
```

In [42]:
```
movies.groupby('Genre')['Gross'].sum().sort_values(ascending=False).head(3)
```

Out[42]:
```
Genre
Drama     3.540997e+10
Action    3.263226e+10
Comedy    1.566387e+10
Name: Gross, dtype: float64
```

find the genre with highest avg IMDB rating

In [36]:
```
movies.groupby('Genre')['IMDB_Rating'].mean().sort_values(ascending=False).head(1)
```

Out[36]:
```
Genre
Western    8.35
Name: IMDB_Rating, dtype: float64
```

find director with most popularity

In [41]:
```
movies.groupby('Director')['No_of_Votes'].sum().sort_values(ascending=False).head(1)
```

Out[41]:
```
Director
Christopher Nolan    11578345
Name: No_of_Votes, dtype: int64
```

find the highest rated movie of each genre

In [43]:
```
movies.groupby('Genre')['IMDB_Rating'].max()
```

Vishal Acharya

Out[43]:
```
Genre
Action       9.0
Adventure    8.6
Animation    8.6
Biography    8.9
Comedy       8.6
Crime        9.2
Drama        9.3
Family       7.8
Fantasy      8.1
Film-Noir    8.1
Horror       8.5
Mystery      8.4
Thriller     7.8
Western      8.8
Name: IMDB_Rating, dtype: float64
```

In [44]:
```
movies.groupby('Genre', as_index=False).agg(
    Highest_Rating=('IMDB_Rating', 'max')
)
```

Vishal Acharya

Out[44]:

	Genre	Highest_Rating
0	Action	9.0
1	Adventure	8.6
2	Animation	8.6
3	Biography	8.9
4	Comedy	8.6
5	Crime	9.2
6	Drama	9.3
7	Family	7.8
8	Fantasy	8.1
9	Film-Noir	8.1
10	Horror	8.5
11	Mystery	8.4
12	Thriller	7.8
13	Western	8.8

In [46]:

```
highest_rated_movies = movies.loc[
    movies.groupby('Genre')['IMDB_Rating'].idxmax()
].reset_index(drop=True)
highest_rated_movies
```

Vishal Acharya

Out[46]:

	Series_Title	Released_Year	Runtime	Genre	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
0	The Dark Knight	2008	152	Action	9.0	Christopher Nolan	Christian Bale	2303232	534858444.0	84.0
1	Interstellar	2014	169	Adventure	8.6	Christopher Nolan	Matthew McConaughey	1512360	188020017.0	74.0
2	Sen to Chihiro no kamikakushi	2001	125	Animation	8.6	Hayao Miyazaki	Daveigh Chase	651376	10055859.0	96.0
3	Schindler's List	1993	195	Biography	8.9	Steven Spielberg	Liam Neeson	1213505	96898818.0	94.0
4	Gisaengchung	2019	132	Comedy	8.6	Bong Joon Ho	Kang-ho Song	552778	53367844.0	96.0
5	The Godfather	1972	175	Crime	9.2	Francis Ford Coppola	Marlon Brando	1620367	134966411.0	100.0
6	The Shawshank Redemption	1994	142	Drama	9.3	Frank Darabont	Tim Robbins	2343110	28341469.0	80.0
7	E.T. the Extra-Terrestrial	1982	115	Family	7.8	Steven Spielberg	Henry Thomas	372490	435110554.0	91.0
8	Das Cabinet des Dr. Caligari	1920	76	Fantasy	8.1	Robert Wiene	Werner Krauss	57428	337574718.0	NaN
9	The Third Man	1949	104	Film-Noir	8.1	Carol Reed	Orson Welles	158731	449191.0	97.0
10	Psycho	1960	109	Horror	8.5	Alfred Hitchcock	Anthony Perkins	604211	32000000.0	97.0
11	Memento	2000	113	Mystery	8.4	Christopher Nolan	Guy Pearce	1125712	25544867.0	80.0
12	Wait Until Dark	1967	108	Thriller	7.8	Terence Young	Audrey Hepburn	27733	17550741.0	81.0

Vishal Acharya

	Series_Title	Released_Year	Runtime	Genre	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
13	Il buono, il brutto, il cattivo	1966	161	Western	8.8	Sergio Leone	Clint Eastwood	688390	6100000.0	90.0

find number of movies done by each actor

In [47]:
```
movies['Star1'].value_counts()
```

Out[47]:
```
Tom Hanks          12
Robert De Niro     11
Clint Eastwood     10
Al Pacino          10
Humphrey Bogart     9
                   ..
Damian Chapa        1
Jean Gabin          1
Susanne Lothar      1
Adrian Molina       1
Chris Sanders       1
Name: Star1, Length: 660, dtype: int64
```

In [48]:
```
movies.groupby('Star1')['Series_Title'].count().sort_values(ascending=False)
```

Out[48]:
```
Star1
Tom Hanks          12
Robert De Niro     11
Clint Eastwood     10
Al Pacino          10
Humphrey Bogart     9
                   ..
Mario Casas         1
Marion Cotillard    1
Marjane Satrapi     1
Martin Freeman      1
Jonathan Pryce      1
Name: Series_Title, Length: 660, dtype: int64
```

Vishal Acharya

find total number of groups -> len

In [49]:
```
len(movies.groupby('Genre'))
```

Out[49]:
```
14
```

In [50]:
```
movies['Genre'].nunique()
```

Out[50]:
```
14
```

find items in each group -> size

In [51]:
```
movies.groupby('Genre').size()
```

Out[51]:
```
Genre
Action       172
Adventure     72
Animation     82
Biography     88
Comedy       155
Crime        107
Drama        289
Family         2
Fantasy        2
Film-Noir      3
Horror        11
Mystery       12
Thriller       1
Western        4
dtype: int64
```

In [53]:
```
genres = movies.groupby('Genre')

genres.nth(0)
```

Vishal Acharya

Out[53]:

Genre	Series_Title	Released_Year	Runtime	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
Action	The Dark Knight	2008	152	9.0	Christopher Nolan	Christian Bale	2303232	534858444.0	84.0
Adventure	Interstellar	2014	169	8.6	Christopher Nolan	Matthew McConaughey	1512360	188020017.0	74.0
Animation	Sen to Chihiro no kamikakushi	2001	125	8.6	Hayao Miyazaki	Daveigh Chase	651376	10055859.0	96.0
Biography	Schindler's List	1993	195	8.9	Steven Spielberg	Liam Neeson	1213505	96898818.0	94.0
Comedy	Gisaengchung	2019	132	8.6	Bong Joon Ho	Kang-ho Song	552778	53367844.0	96.0
Crime	The Godfather	1972	175	9.2	Francis Ford Coppola	Marlon Brando	1620367	134966411.0	100.0
Drama	The Shawshank Redemption	1994	142	9.3	Frank Darabont	Tim Robbins	2343110	28341469.0	80.0
Family	E.T. the Extra-Terrestrial	1982	115	7.8	Steven Spielberg	Henry Thomas	372490	435110554.0	91.0
Fantasy	Das Cabinet des Dr. Caligari	1920	76	8.1	Robert Wiene	Werner Krauss	57428	337574718.0	NaN
Film-Noir	The Third Man	1949	104	8.1	Carol Reed	Orson Welles	158731	449191.0	97.0
Horror	Psycho	1960	109	8.5	Alfred Hitchcock	Anthony Perkins	604211	32000000.0	97.0
Mystery	Memento	2000	113	8.4	Christopher Nolan	Guy Pearce	1125712	25544867.0	80.0
Thriller	Wait Until Dark	1967	108	7.8	Terence Young	Audrey Hepburn	27733	17550741.0	81.0

Genre	Series_Title	Released_Year	Runtime	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
Western	Il buono, il brutto, il cattivo	1966	161	8.8	Sergio Leone	Clint Eastwood	688390	6100000.0	90.0

In [54]:
```
genres.nth(-1)
```

Vishal Acharya

Vishal Acharya

Out[54]:

Genre	Series_Title	Released_Year	Runtime	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
Action	Escape from Alcatraz	1979	112	7.6	Don Siegel	Clint Eastwood	121731	43000000.0	76.0
Adventure	Kelly's Heroes	1970	144	7.6	Brian G. Hutton	Clint Eastwood	45338	1378435.0	50.0
Animation	The Jungle Book	1967	78	7.6	Wolfgang Reitherman	Phil Harris	166409	141843612.0	65.0
Biography	Midnight Express	1978	121	7.6	Alan Parker	Brad Davis	73662	35000000.0	59.0
Comedy	Breakfast at Tiffany's	1961	115	7.6	Blake Edwards	Audrey Hepburn	166544	679874270.0	76.0
Crime	The 39 Steps	1935	86	7.6	Alfred Hitchcock	Robert Donat	51853	302787539.0	93.0
Drama	Lifeboat	1944	97	7.6	Alfred Hitchcock	Tallulah Bankhead	26471	852142728.0	78.0
Family	Willy Wonka & the Chocolate Factory	1971	100	7.8	Mel Stuart	Gene Wilder	178731	4000000.0	67.0
Fantasy	Nosferatu	1922	94	7.9	F.W. Murnau	Max Schreck	88794	445151978.0	NaN
Film-Noir	Shadow of a Doubt	1943	108	7.8	Alfred Hitchcock	Teresa Wright	59556	123353292.0	94.0
Horror	The Others	2001	101	7.6	Alejandro Amenábar	Nicole Kidman	337651	96522687.0	74.0
Mystery	Lost Highway	1997	134	7.6	David Lynch	Bill Pullman	131101	3796699.0	52.0
Thriller	Wait Until Dark	1967	108	7.8	Terence Young	Audrey Hepburn	27733	17550741.0	81.0
Western	The Outlaw Josey Wales	1976	135	7.8	Clint Eastwood	Clint Eastwood	65659	31800000.0	69.0

Vishal Acharya

In [62]:
```
genres.nth(6)
```

Out[62]:

Genre	Series_Title	Released_Year	Runtime	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
Action	Star Wars: Episode V - The Empire Strikes Back	1980	124	8.7	Irvin Kershner	Mark Hamill	1159315	290475067.0	82.0
Adventure	North by Northwest	1959	136	8.3	Alfred Hitchcock	Cary Grant	299198	13275000.0	98.0
Animation	WALL·E	2008	98	8.4	Andrew Stanton	Ben Burtt	999790	223808164.0	95.0
Biography	Braveheart	1995	178	8.3	Mel Gibson	Mel Gibson	959181	75600000.0	68.0
Comedy	The Great Dictator	1940	125	8.4	Charles Chaplin	Charles Chaplin	203150	288475.0	NaN
Crime	Se7en	1995	127	8.6	David Fincher	Morgan Freeman	1445096	100125643.0	65.0
Drama	It's a Wonderful Life	1946	130	8.6	Frank Capra	James Stewart	405801	82385199.0	89.0
Horror	Get Out	2017	104	7.7	Jordan Peele	Daniel Kaluuya	492851	176040665.0	85.0
Mystery	Sleuth	1972	138	8.0	Joseph L. Mankiewicz	Laurence Olivier	44748	4081254.0	NaN

In [55]:
```
movies['Genre'].value_counts()
```

Vishal Acharya

Out[55]:
```
Drama        289
Action       172
Comedy       155
Crime        107
Biography     88
Animation     82
Adventure     72
Mystery       12
Horror        11
Western        4
Film-Noir      3
Family         2
Fantasy        2
Thriller       1
Name: Genre, dtype: int64
```

get_group -> vs filtering

In [56]:
```
genres.get_group('Fantasy')
```

Out[56]:

	Series_Title	Released_Year	Runtime	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
321	Das Cabinet des Dr. Caligari	1920	76	8.1	Robert Wiene	Werner Krauss	57428	337574718.0	NaN
568	Nosferatu	1922	94	7.9	F.W. Murnau	Max Schreck	88794	445151978.0	NaN

In [57]:
```
movies[movies['Genre'] == 'Fantasy']
```

Out[57]:

	Series_Title	Released_Year	Runtime	Genre	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
321	Das Cabinet des Dr. Caligari	1920	76	Fantasy	8.1	Robert Wiene	Werner Krauss	57428	337574718.0	NaN
568	Nosferatu	1922	94	Fantasy	7.9	F.W. Murnau	Max Schreck	88794	445151978.0	NaN

groups

Vishal Acharya

In [58]:
```
genres.groups
```

Out[58]:
```
{'Action': [2, 5, 8, 10, 13, 14, 16, 29, 30, 31, 39, 42, 44, 55, 57, 59, 60, 63, 68, 72, 106, 109, 129, 130, 134, 140, 142, 1
44, 152, 155, 160, 161, 166, 168, 171, 172, 177, 181, 194, 201, 202, 216, 217, 223, 224, 236, 241, 262, 275, 294, 308, 320, 3
25, 326, 331, 337, 339, 340, 343, 345, 348, 351, 353, 356, 357, 362, 368, 369, 375, 376, 390, 410, 431, 436, 473, 477, 479, 4
82, 488, 493, 496, 502, 507, 511, 532, 535, 540, 543, 564, 569, 570, 573, 577, 582, 583, 602, 605, 608, 615, 623, ...], 'Adve
nture': [21, 47, 93, 110, 114, 116, 118, 137, 178, 179, 191, 193, 209, 226, 231, 247, 267, 273, 281, 300, 301, 304, 306, 323,
329, 361, 366, 377, 402, 406, 415, 426, 458, 470, 497, 498, 506, 513, 514, 537, 549, 552, 553, 566, 576, 604, 609, 618, 638,
647, 675, 681, 686, 692, 711, 713, 739, 755, 781, 797, 798, 851, 873, 884, 912, 919, 947, 957, 964, 966, 984, 991], 'Animatio
n': [23, 43, 46, 56, 58, 61, 66, 70, 101, 135, 146, 151, 158, 170, 197, 205, 211, 213, 219, 229, 230, 242, 245, 246, 270, 33
0, 332, 358, 367, 378, 386, 389, 394, 395, 399, 401, 405, 409, 469, 499, 510, 516, 518, 522, 578, 586, 592, 595, 596, 599, 63
3, 640, 643, 651, 665, 672, 694, 728, 740, 741, 744, 756, 758, 761, 771, 783, 796, 799, 822, 828, 843, 875, 891, 892, 902, 90
6, 920, 956, 971, 976, 986, 992], 'Biography': [7, 15, 18, 35, 38, 54, 102, 107, 131, 139, 147, 157, 159, 173, 176, 212, 215,
218, 228, 235, 243, 263, 276, 282, 290, 298, 317, 328, 338, 342, 346, 359, 360, 365, 372, 373, 385, 411, 416, 418, 424, 429,
484, 525, 536, 542, 545, 575, 579, 587, 600, 606, 614, 622, 632, 635, 644, 649, 650, 657, 671, 673, 684, 729, 748, 753, 757,
759, 766, 770, 779, 809, 810, 815, 820, 831, 849, 858, 877, 882, 897, 910, 915, 923, 940, 949, 952, 987], 'Comedy': [19, 26,
51, 52, 64, 78, 83, 95, 96, 112, 117, 120, 127, 128, 132, 153, 169, 183, 192, 204, 207, 208, 214, 221, 233, 238, 240, 250, 25
1, 252, 256, 261, 266, 277, 284, 311, 313, 316, 318, 322, 327, 374, 379, 381, 392, 396, 403, 413, 414, 417, 427, 435, 445, 44
6, 449, 455, 459, 460, 463, 464, 466, 471, 472, 475, 481, 490, 494, 500, 503, 509, 526, 528, 530, 531, 533, 538, 539, 541, 54
7, 557, 558, 562, 563, 565, 574, 591, 593, 594, 598, 613, 626, 630, 660, 662, 667, 679, 680, 683, 687, 701, ...], 'Crime':
[1, 3, 4, 6, 22, 25, 27, 28, 33, 37, 41, 71, 77, 79, 86, 87, 103, 108, 111, 113, 123, 125, 133, 136, 162, 163, 164, 165, 180,
186, 187, 189, 198, 222, 232, 239, 255, 257, 287, 288, 299, 305, 335, 363, 364, 380, 384, 397, 437, 438, 441, 442, 444, 450,
451, 465, 474, 480, 485, 487, 505, 512, 519, 520, 523, 527, 546, 556, 560, 584, 597, 603, 607, 611, 621, 639, 653, 664, 669,
676, 695, 708, 723, 762, 763, 767, 775, 791, 795, 802, 811, 823, 827, 833, 885, 895, 921, 922, 926, 938, ...], 'Drama': [0,
9, 11, 17, 20, 24, 32, 34, 36, 40, 45, 50, 53, 62, 65, 67, 73, 74, 76, 80, 82, 84, 85, 88, 89, 90, 91, 92, 94, 97, 98, 99, 10
0, 104, 105, 121, 122, 124, 126, 138, 141, 143, 148, 149, 150, 154, 156, 167, 174, 175, 182, 184, 185, 188, 190, 195, 196, 19
9, 200, 203, 206, 210, 225, 227, 234, 237, 244, 248, 249, 253, 254, 258, 259, 260, 264, 265, 268, 269, 272, 274, 278, 279, 28
0, 283, 285, 286, 289, 291, 292, 293, 295, 296, 297, 302, 303, 307, 310, 312, 314, 315, ...], 'Family': [688, 698], 'Fantas
y': [321, 568], 'Film-Noir': [309, 456, 712], 'Horror': [49, 75, 271, 419, 544, 707, 724, 844, 876, 932, 948], 'Mystery': [6
9, 81, 119, 145, 220, 393, 420, 714, 829, 899, 959, 961], 'Thriller': [700], 'Western': [12, 48, 115, 691]}
```

describe

In [59]:
```
genres.describe()
```

Vishal Acharya

Out[59]:

	Runtime								IMDB_Rating		...		Gross		
	count	mean	std	min	25%	50%	75%	max	count	mean	...	75%	max	count	
Genre															
Action	172.0	129.046512	28.500706	45.0	110.75	127.5	143.25	321.0	172.0	7.949419	...	2.674437e+08	936662225.0	143.0	73.4
Adventure	72.0	134.111111	33.317320	88.0	109.00	127.0	149.00	228.0	72.0	7.937500	...	1.998070e+08	874211619.0	64.0	78.4
Animation	82.0	99.585366	14.530471	71.0	90.00	99.5	106.75	137.0	82.0	7.930488	...	2.520612e+08	873839108.0	75.0	81.0
Biography	88.0	136.022727	25.514466	93.0	120.00	129.0	146.25	209.0	88.0	7.938636	...	9.829924e+07	753585104.0	79.0	76.2
Comedy	155.0	112.129032	22.946213	68.0	96.00	106.0	124.50	188.0	155.0	7.901290	...	8.107809e+07	886752933.0	125.0	78.7
Crime	107.0	126.392523	27.689231	80.0	106.50	122.0	141.50	229.0	107.0	8.016822	...	7.102163e+07	790482117.0	87.0	77.0
Drama	289.0	124.737024	27.740490	64.0	105.00	121.0	137.00	242.0	289.0	7.957439	...	1.164461e+08	924558264.0	241.0	79.7
Family	2.0	107.500000	10.606602	100.0	103.75	107.5	111.25	115.0	2.0	7.800000	...	3.273329e+08	435110554.0	2.0	79.0
Fantasy	2.0	85.000000	12.727922	76.0	80.50	85.0	89.50	94.0	2.0	8.000000	...	4.182577e+08	445151978.0	0.0	
Film-Noir	3.0	104.000000	4.000000	100.0	102.00	104.0	106.00	108.0	3.0	7.966667	...	6.273068e+07	123353292.0	3.0	95.6
Horror	11.0	102.090909	13.604812	71.0	98.00	103.0	109.00	122.0	11.0	7.909091	...	1.362817e+08	298791505.0	11.0	80.0
Mystery	12.0	119.083333	14.475423	96.0	110.75	117.5	130.25	138.0	12.0	7.975000	...	1.310949e+08	474203697.0	8.0	79.1
Thriller	1.0	108.000000	NaN	108.0	108.00	108.0	108.00	108.0	1.0	7.800000	...	1.755074e+07	17550741.0	1.0	81.0
Western	4.0	148.250000	17.153717	132.0	134.25	148.0	162.00	165.0	4.0	8.350000	...	1.920000e+07	31800000.0	4.0	78.2

14 rows × 40 columns

sample

In [60]:
```
genres.sample(2,replace=True)
```

Vishal Acharya

Out[60]:

	Series_Title	Released_Year	Runtime	Genre	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
507	Ying xiong	2002	120	Action	7.9	Yimou Zhang	Jet Li	173999	53710019.0	85.0
774	3:10 to Yuma	2007	122	Action	7.7	James Mangold	Russell Crowe	288797	53606916.0	76.0
470	Hunt for the Wilderpeople	2016	101	Adventure	7.9	Taika Waititi	Sam Neill	111483	5202582.0	81.0
537	La montaña sagrada	1973	114	Adventure	7.9	Alejandro Jodorowsky	Alejandro Jodorowsky	37183	61001.0	76.0
409	Tenkû no shiro Rapyuta	1986	125	Animation	8.0	Hayao Miyazaki	Mayumi Tanaka	150140	715920290.0	78.0
405	Akira	1988	124	Animation	8.0	Katsuhiro Ôtomo	Mitsuo Iwata	164918	553171.0	NaN
360	The Pursuit of Happyness	2006	117	Biography	8.0	Gabriele Muccino	Will Smith	448930	163566459.0	64.0
228	Hachi: A Dog's Tale	2009	93	Biography	8.1	Lasse Hallström	Richard Gere	253575	91090444.0	NaN
251	Crna macka, beli macor	1998	127	Comedy	8.1	Emir Kusturica	Bajram Severdzan	50862	348660.0	73.0
311	The Shop Around the Corner	1940	99	Comedy	8.1	Ernst Lubitsch	Margaret Sullavan	28450	203300.0	96.0
287	Le samouraï	1967	105	Crime	8.1	Jean-Pierre Melville	Alain Delon	45434	39481.0	NaN
133	Talvar	2015	132	Crime	8.2	Meghna Gulzar	Irrfan Khan	31142	342370.0	NaN
941	25th Hour	2002	135	Drama	7.6	Spike Lee	Edward Norton	169708	13060843.0	68.0

Vishal Acharya

	Series_Title	Released_Year	Runtime	Genre	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
249	La leggenda del pianista sull'oceano	1998	169	Drama	8.1	Giuseppe Tornatore	Tim Roth	59020	259127.0	58.0
698	Willy Wonka & the Chocolate Factory	1971	100	Family	7.8	Mel Stuart	Gene Wilder	178731	4000000.0	67.0
698	Willy Wonka & the Chocolate Factory	1971	100	Family	7.8	Mel Stuart	Gene Wilder	178731	4000000.0	67.0
321	Das Cabinet des Dr. Caligari	1920	76	Fantasy	8.1	Robert Wiene	Werner Krauss	57428	337574718.0	NaN
321	Das Cabinet des Dr. Caligari	1920	76	Fantasy	8.1	Robert Wiene	Werner Krauss	57428	337574718.0	NaN
309	The Third Man	1949	104	Film-Noir	8.1	Carol Reed	Orson Welles	158731	449191.0	97.0
309	The Third Man	1949	104	Film-Noir	8.1	Carol Reed	Orson Welles	158731	449191.0	97.0
948	The Others	2001	101	Horror	7.6	Alejandro Amenábar	Nicole Kidman	337651	96522687.0	74.0
948	The Others	2001	101	Horror	7.6	Alejandro Amenábar	Nicole Kidman	337651	96522687.0	74.0
959	Dark City	1998	100	Mystery	7.6	Alex Proyas	Rufus Sewell	187927	14378331.0	66.0
220	Kahaani	2012	122	Mystery	8.1	Sujoy Ghosh	Vidya Balan	57806	1035953.0	NaN
700	Wait Until Dark	1967	108	Thriller	7.8	Terence Young	Audrey Hepburn	27733	17550741.0	81.0
700	Wait Until Dark	1967	108	Thriller	7.8	Terence Young	Audrey Hepburn	27733	17550741.0	81.0

Vishal Acharya

	Series_Title	Released_Year	Runtime	Genre	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
48	Once Upon a Time in the West	1968	165	Western	8.5	Sergio Leone	Henry Fonda	302844	5321508.0	80.0
115	Per qualche dollaro in più	1965	132	Western	8.3	Sergio Leone	Clint Eastwood	232772	15000000.0	74.0

nunique

In [61]:
```
genres.nunique()
```

Vishal Acharya

Out[61]:

Genre	Series_Title	Released_Year	Runtime	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
Action	172	61	78	15	123	121	172	172	50
Adventure	72	49	58	10	59	59	72	72	33
Animation	82	35	41	11	51	77	82	82	29
Biography	88	44	56	13	76	72	88	88	40
Comedy	155	72	70	11	113	133	155	155	44
Crime	106	56	65	14	86	85	107	107	39
Drama	289	83	95	14	211	250	288	287	52
Family	2	2	2	1	2	2	2	2	2
Fantasy	2	2	2	2	2	2	2	2	0
Film-Noir	3	3	3	3	3	3	3	3	3
Horror	11	11	10	8	10	11	11	11	9
Mystery	12	11	10	8	10	11	12	12	7
Thriller	1	1	1	1	1	1	1	1	1
Western	4	4	4	4	2	2	4	4	4

agg method: passing dict

In [63]:

```
genres.agg(
    {
        'Runtime':'mean',
        'IMDB_Rating':'mean',
        'No_of_Votes':'sum',
        'Gross':'sum',
        'Metascore':'min'
```

Vishal Acharya

```
    }
)
```

Out[63]:

Genre	Runtime	IMDB_Rating	No_of_Votes	Gross	Metascore
Action	129.046512	7.949419	72282412	3.263226e+10	33.0
Adventure	134.111111	7.937500	22576163	9.496922e+09	41.0
Animation	99.585366	7.930488	21978630	1.463147e+10	61.0
Biography	136.022727	7.938636	24006844	8.276358e+09	48.0
Comedy	112.129032	7.901290	27620327	1.566387e+10	45.0
Crime	126.392523	8.016822	33533615	8.452632e+09	47.0
Drama	124.737024	7.957439	61367304	3.540997e+10	28.0
Family	107.500000	7.800000	551221	4.391106e+08	67.0
Fantasy	85.000000	8.000000	146222	7.827267e+08	NaN
Film-Noir	104.000000	7.966667	367215	1.259105e+08	94.0
Horror	102.090909	7.909091	3742556	1.034649e+09	46.0
Mystery	119.083333	7.975000	4203004	1.256417e+09	52.0
Thriller	108.000000	7.800000	27733	1.755074e+07	81.0
Western	148.250000	8.350000	1289665	5.822151e+07	69.0

passing list

In [64]:

```
genres.agg(['min','max','mean','sum'])
```

Vishal Acharya

Out[64]:

	Runtime				IMDB_Rating				No_of_Votes					
	min	max	mean	sum	min	max	mean	sum	min	max	mean	sum	min	m
Genre														
Action	45	321	129.046512	22196	7.6	9.0	7.949419	1367.3	25312	2303232	420246.581395	72282412	3296.0	93666222
Adventure	88	228	134.111111	9656	7.6	8.6	7.937500	571.5	29999	1512360	313557.819444	22576163	61001.0	87421161
Animation	71	137	99.585366	8166	7.6	8.6	7.930488	650.3	25229	999790	268032.073171	21978630	128985.0	87383910
Biography	93	209	136.022727	11970	7.6	8.9	7.938636	698.6	27254	1213505	272805.045455	24006844	21877.0	75358510
Comedy	68	188	112.129032	17380	7.6	8.6	7.901290	1224.7	26337	939631	178195.658065	27620327	1305.0	88675293
Crime	80	229	126.392523	13524	7.6	9.2	8.016822	857.8	27712	1826188	313398.271028	33533615	6013.0	79048211
Drama	64	242	124.737024	36049	7.6	9.3	7.957439	2299.7	25088	2343110	212343.612457	61367304	3600.0	92455826
Family	100	115	107.500000	215	7.8	7.8	7.800000	15.6	178731	372490	275610.500000	551221	4000000.0	43511055
Fantasy	76	94	85.000000	170	7.9	8.1	8.000000	16.0	57428	88794	73111.000000	146222	337574718.0	44515197
Film-Noir	100	108	104.000000	312	7.8	8.1	7.966667	23.9	59556	158731	122405.000000	367215	449191.0	12335329
Horror	71	122	102.090909	1123	7.6	8.5	7.909091	87.0	27007	787806	340232.363636	3742556	89029.0	29879150
Mystery	96	138	119.083333	1429	7.6	8.4	7.975000	95.7	33982	1129894	350250.333333	4203004	1035953.0	47420369
Thriller	108	108	108.000000	108	7.8	7.8	7.800000	7.8	27733	27733	27733.000000	27733	17550741.0	1755074
Western	132	165	148.250000	593	7.8	8.8	8.350000	33.4	65659	688390	322416.250000	1289665	5321508.0	3180000

Adding both the syntax

In [65]:

```
genres.agg(
    {
        'Runtime':['min','mean'],
        'IMDB_Rating':'mean',
        'No_of_Votes':['sum','max'],
        'Gross':'sum',
```

Vishal Acharya

```
        'Metascore':'min'
    }
)
```

Out[65]:

	Runtime		IMDB_Rating	No_of_Votes		Gross	Metascore
	min	mean	mean	sum	max	sum	min
Genre							
Action	45	129.046512	7.949419	72282412	2303232	3.263226e+10	33.0
Adventure	88	134.111111	7.937500	22576163	1512360	9.496922e+09	41.0
Animation	71	99.585366	7.930488	21978630	999790	1.463147e+10	61.0
Biography	93	136.022727	7.938636	24006844	1213505	8.276358e+09	48.0
Comedy	68	112.129032	7.901290	27620327	939631	1.566387e+10	45.0
Crime	80	126.392523	8.016822	33533615	1826188	8.452632e+09	47.0
Drama	64	124.737024	7.957439	61367304	2343110	3.540997e+10	28.0
Family	100	107.500000	7.800000	551221	372490	4.391106e+08	67.0
Fantasy	76	85.000000	8.000000	146222	88794	7.827267e+08	NaN
Film-Noir	100	104.000000	7.966667	367215	158731	1.259105e+08	94.0
Horror	71	102.090909	7.909091	3742556	787806	1.034649e+09	46.0
Mystery	96	119.083333	7.975000	4203004	1129894	1.256417e+09	52.0
Thriller	108	108.000000	7.800000	27733	27733	1.755074e+07	81.0
Western	132	148.250000	8.350000	1289665	688390	5.822151e+07	69.0

looping on groups: Find each group display which movie had highest rating

In [66]:

```
df = pd.DataFrame(columns=movies.columns)
for group,data in genres:
  df = df.append(data[data['IMDB_Rating'] == data['IMDB_Rating'].max()])
```

```
df
```

Vishal Acharya

Vishal Acharya

Out[66]:

	Series_Title	Released_Year	Runtime	Genre	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
2	The Dark Knight	2008	152	Action	9.0	Christopher Nolan	Christian Bale	2303232	534858444.0	84.0
21	Interstellar	2014	169	Adventure	8.6	Christopher Nolan	Matthew McConaughey	1512360	188020017.0	74.0
23	Sen to Chihiro no kamikakushi	2001	125	Animation	8.6	Hayao Miyazaki	Daveigh Chase	651376	10055859.0	96.0
7	Schindler's List	1993	195	Biography	8.9	Steven Spielberg	Liam Neeson	1213505	96898818.0	94.0
19	Gisaengchung	2019	132	Comedy	8.6	Bong Joon Ho	Kang-ho Song	552778	53367844.0	96.0
26	La vita è bella	1997	116	Comedy	8.6	Roberto Benigni	Roberto Benigni	623629	57598247.0	59.0
1	The Godfather	1972	175	Crime	9.2	Francis Ford Coppola	Marlon Brando	1620367	134966411.0	100.0
0	The Shawshank Redemption	1994	142	Drama	9.3	Frank Darabont	Tim Robbins	2343110	28341469.0	80.0
688	E.T. the Extra-Terrestrial	1982	115	Family	7.8	Steven Spielberg	Henry Thomas	372490	435110554.0	91.0
698	Willy Wonka & the Chocolate Factory	1971	100	Family	7.8	Mel Stuart	Gene Wilder	178731	4000000.0	67.0
321	Das Cabinet des Dr. Caligari	1920	76	Fantasy	8.1	Robert Wiene	Werner Krauss	57428	337574718.0	NaN
309	The Third Man	1949	104	Film-Noir	8.1	Carol Reed	Orson Welles	158731	449191.0	97.0

Vishal Acharya

	Series_Title	Released_Year	Runtime	Genre	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
49	Psycho	1960	109	Horror	8.5	Alfred Hitchcock	Anthony Perkins	604211	32000000.0	97.0
69	Memento	2000	113	Mystery	8.4	Christopher Nolan	Guy Pearce	1125712	25544867.0	80.0
81	Rear Window	1954	112	Mystery	8.4	Alfred Hitchcock	James Stewart	444074	36764313.0	100.0
700	Wait Until Dark	1967	108	Thriller	7.8	Terence Young	Audrey Hepburn	27733	17550741.0	81.0
12	Il buono, il brutto, il cattivo	1966	161	Western	8.8	Sergio Leone	Clint Eastwood	688390	6100000.0	90.0

split (apply) combine

In [67]:
```
genres.apply(min)
```

Vishal Acharya

Out[67]:

Genre	Series_Title	Released_Year	Runtime	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
Action	300	1924	45	7.6	Abhishek Chaubey	Aamir Khan	25312	3296.0	33.0
Adventure	2001: A Space Odyssey	1925	88	7.6	Akira Kurosawa	Aamir Khan	29999	61001.0	41.0
Animation	Akira	1940	71	7.6	Adam Elliot	Adrian Molina	25229	128985.0	61.0
Biography	12 Years a Slave	1928	93	7.6	Adam McKay	Adrien Brody	27254	21877.0	48.0
Comedy	(500) Days of Summer	1921	68	7.6	Alejandro G. Iñárritu	Aamir Khan	26337	1305.0	45.0
Crime	12 Angry Men	1931	80	7.6	Akira Kurosawa	Ajay Devgn	27712	6013.0	47.0
Drama	1917	1925	64	7.6	Aamir Khan	Abhay Deol	25088	3600.0	28.0
Family	E.T. the Extra-Terrestrial	1971	100	7.8	Mel Stuart	Gene Wilder	178731	4000000.0	67.0
Fantasy	Das Cabinet des Dr. Caligari	1920	76	7.9	F.W. Murnau	Max Schreck	57428	337574718.0	NaN
Film-Noir	Shadow of a Doubt	1941	100	7.8	Alfred Hitchcock	Humphrey Bogart	59556	449191.0	94.0
Horror	Alien	1933	71	7.6	Alejandro Amenábar	Anthony Perkins	27007	89029.0	46.0
Mystery	Dark City	1938	96	7.6	Alex Proyas	Bernard-Pierre Donnadieu	33982	1035953.0	52.0
Thriller	Wait Until Dark	1967	108	7.8	Terence Young	Audrey Hepburn	27733	17550741.0	81.0
Western	Il buono, il brutto, il cattivo	1965	132	7.8	Clint Eastwood	Clint Eastwood	65659	5321508.0	69.0

Vishal Acharya

find number of movies starting with A for each group

In [68]:
```
def foo(group):
  return group['Series_Title'].str.startswith('A').sum()
genres.apply(foo)
```

Out[68]:
```
Genre
Action       10
Adventure     2
Animation     2
Biography     9
Comedy       14
Crime         4
Drama        21
Family        0
Fantasy       0
Film-Noir     0
Horror        1
Mystery       0
Thriller      0
Western       0
dtype: int64
```

find ranking of each movie in the group according to IMDB score

In [69]:
```
def rank_movie(group):
  group['genre_rank'] = group['IMDB_Rating'].rank(ascending=False)
  return group
genres.apply(rank_movie)
```

Vishal Acharya

Out[69]:

	Series_Title	Released_Year	Runtime	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore	genre_rank
0	The Shawshank Redemption	1994	142	9.3	Frank Darabont	Tim Robbins	2343110	28341469.0	80.0	1.0
1	The Godfather	1972	175	9.2	Francis Ford Coppola	Marlon Brando	1620367	134966411.0	100.0	1.0
2	The Dark Knight	2008	152	9.0	Christopher Nolan	Christian Bale	2303232	534858444.0	84.0	1.0
3	The Godfather: Part II	1974	202	9.0	Francis Ford Coppola	Al Pacino	1129952	57300000.0	90.0	2.5
4	12 Angry Men	1957	96	9.0	Sidney Lumet	Henry Fonda	689845	4360000.0	96.0	2.5
...	...	...	...	...	...	...	...	...	...	...
995	Breakfast at Tiffany's	1961	115	7.6	Blake Edwards	Audrey Hepburn	166544	679874270.0	76.0	147.0
996	Giant	1956	201	7.6	George Stevens	Elizabeth Taylor	34075	195217415.0	84.0	272.5
997	From Here to Eternity	1953	118	7.6	Fred Zinnemann	Burt Lancaster	43374	30500000.0	85.0	272.5
998	Lifeboat	1944	97	7.6	Alfred Hitchcock	Tallulah Bankhead	26471	852142728.0	78.0	272.5
999	The 39 Steps	1935	86	7.6	Alfred Hitchcock	Robert Donat	51853	302787539.0	93.0	101.0

1000 rows × 10 columns

find normalized IMDB rating group wise

In [70]:

```
def normal(group):
  group['norm_rating'] = (group['IMDB_Rating'] - group['IMDB_Rating'].min())/(group['IMDB_Rating'].max() - group['IMDB_Rating'
```

Vishal Acharya

```
    return group

genres.apply(normal)
```

Out[70]:

	Series_Title	Released_Year	Runtime	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore	norm_rating
0	The Shawshank Redemption	1994	142	9.3	Frank Darabont	Tim Robbins	2343110	28341469.0	80.0	1.000
1	The Godfather	1972	175	9.2	Francis Ford Coppola	Marlon Brando	1620367	134966411.0	100.0	1.000
2	The Dark Knight	2008	152	9.0	Christopher Nolan	Christian Bale	2303232	534858444.0	84.0	1.000
3	The Godfather: Part II	1974	202	9.0	Francis Ford Coppola	Al Pacino	1129952	57300000.0	90.0	0.875
4	12 Angry Men	1957	96	9.0	Sidney Lumet	Henry Fonda	689845	4360000.0	96.0	0.875
...	...	...	...	...	...	...	...	...	...	...
995	Breakfast at Tiffany's	1961	115	7.6	Blake Edwards	Audrey Hepburn	166544	679874270.0	76.0	0.000
996	Giant	1956	201	7.6	George Stevens	Elizabeth Taylor	34075	195217415.0	84.0	0.000
997	From Here to Eternity	1953	118	7.6	Fred Zinnemann	Burt Lancaster	43374	30500000.0	85.0	0.000
998	Lifeboat	1944	97	7.6	Alfred Hitchcock	Tallulah Bankhead	26471	852142728.0	78.0	0.000
999	The 39 Steps	1935	86	7.6	Alfred Hitchcock	Robert Donat	51853	302787539.0	93.0	0.000

1000 rows × 10 columns

Vishal Acharya

groupby on multiple cols

In [71]:
```
duo = movies.groupby(['Director','Star1'])
duo
```

Out[71]:
```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x000002892701DBE0>
```

size

In [72]:
```
duo.size()
```

Out[72]:
```
Director             Star1
Aamir Khan           Amole Gupte       1
Aaron Sorkin         Eddie Redmayne    1
Abdellatif Kechiche  Léa Seydoux       1
Abhishek Chaubey     Shahid Kapoor     1
Abhishek Kapoor      Amit Sadh         1
                                      ..
Zaza Urushadze       Lembit Ulfsak     1
Zoya Akhtar          Hrithik Roshan    1
                     Vijay Varma       1
Çagan Irmak          Çetin Tekindor    1
Ömer Faruk Sorak     Cem Yilmaz        1
Length: 898, dtype: int64
```

get_group

In [73]:
```
duo.get_group(('Aamir Khan','Amole Gupte'))
```

Out[73]:

	Series_Title	Released_Year	Runtime	Genre	IMDB_Rating	Director	Star1	No_of_Votes	Gross	Metascore
65	Taare Zameen Par	2007	165	Drama	8.4	Aamir Khan	Amole Gupte	168895	1223869.0	NaN

find the most earning actor->director combo

In [74]:
```
duo['Gross'].sum().sort_values(ascending=False).head(1)
```

Vishal Acharya

Out[74]:
```
Director        Star1
Akira Kurosawa  Toshirô Mifune    2.999877e+09
Name: Gross, dtype: float64
```

find the best(in-terms of metascore(avg)) actor->genre combo

In [75]:
```
movies.groupby(['Star1','Genre'])['Metascore'].mean().reset_index().sort_values('Metascore',ascending=False).head(1)
```

Out[75]:

	Star1	Genre	Metascore
230	Ellar Coltrane	Drama	100.0

agg on multiple groupby

In [76]:
```
duo.agg(['min','max','mean'])
```

Vishal Acharya

Out[76]:

		Runtime			IMDB_Rating			No_of_Votes			Gross			Me	
		min	max	mean	min	max	mean	min	max	mean	min	max	mean	min	max
Director	**Star1**														
Aamir Khan	**Amole Gupte**	165	165	165.0	8.4	8.4	8.4	168895	168895	168895.0	1223869.0	1223869.0	1223869.0	NaN	NaN
Aaron Sorkin	**Eddie Redmayne**	129	129	129.0	7.8	7.8	7.8	89896	89896	89896.0	853090410.0	853090410.0	853090410.0	77.0	77.0
Abdellatif Kechiche	**Léa Seydoux**	180	180	180.0	7.7	7.7	7.7	138741	138741	138741.0	2199675.0	2199675.0	2199675.0	89.0	89.0
Abhishek Chaubey	**Shahid Kapoor**	148	148	148.0	7.8	7.8	7.8	27175	27175	27175.0	218428303.0	218428303.0	218428303.0	NaN	NaN
Abhishek Kapoor	**Amit Sadh**	130	130	130.0	7.7	7.7	7.7	32628	32628	32628.0	1122527.0	1122527.0	1122527.0	40.0	40.0
...	**...**	...	...	...	...	...	...	...	...	...	...	...	...	...	...
Zaza Urushadze	**Lembit Ulfsak**	87	87	87.0	8.2	8.2	8.2	40382	40382	40382.0	144501.0	144501.0	144501.0	73.0	73.0
Zoya Akhtar	**Hrithik Roshan**	155	155	155.0	8.1	8.1	8.1	67927	67927	67927.0	3108485.0	3108485.0	3108485.0	NaN	NaN
	Vijay Varma	154	154	154.0	8.0	8.0	8.0	31886	31886	31886.0	5566534.0	5566534.0	5566534.0	65.0	65.0
Çagan Irmak	**Çetin Tekindor**	112	112	112.0	8.3	8.3	8.3	78925	78925	78925.0	461855363.0	461855363.0	461855363.0	NaN	NaN
Ömer Faruk Sorak	**Cem Yilmaz**	127	127	127.0	8.0	8.0	8.0	56960	56960	56960.0	196206077.0	196206077.0	196206077.0	NaN	NaN

898 rows × 15 columns

Vishal Acharya

TASK-2

In [77]:
```
ipl = pd.read_csv('deliveries.csv')
ipl.head()
```

Out[77]:

	match_id	inning	batting_team	bowling_team	over	ball	batsman	non_striker	bowler	is_super_over	...	bye_runs	legbye_runs	no
0	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	1	DA Warner	S Dhawan	TS Mills	0	...	0	0	
1	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	2	DA Warner	S Dhawan	TS Mills	0	...	0	0	
2	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	3	DA Warner	S Dhawan	TS Mills	0	...	0	0	
3	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	4	DA Warner	S Dhawan	TS Mills	0	...	0	0	
4	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	5	DA Warner	S Dhawan	TS Mills	0	...	0	0	

5 rows × 21 columns

In [78]:
```
ipl.shape
```

Out[78]:
```
(179078, 21)
```

find the top 10 batsman in terms of runs

In [79]:
```
ipl.groupby('batsman')['batsman_runs'].sum().sort_values(ascending=False).head(10)
```

Vishal Acharya

Out[79]:
```
batsman
V Kohli           5434
SK Raina          5415
RG Sharma         4914
DA Warner         4741
S Dhawan          4632
CH Gayle          4560
MS Dhoni          4477
RV Uthappa        4446
AB de Villiers    4428
G Gambhir         4223
Name: batsman_runs, dtype: int64
```

find the batsman with max no of sixes

In [80]:
```
six = ipl[ipl['batsman_runs'] == 6]

six.groupby('batsman')['batsman'].count().sort_values(ascending=False).head(1).index[0]
```

Out[80]:
```
'CH Gayle'
```

find batsman with most number of 4's and 6's in last 5 overs

In [81]:
```
temp_df = ipl[ipl['over'] > 15]
temp_df = temp_df[(temp_df['batsman_runs'] == 4) | (temp_df['batsman_runs'] == 6)]
temp_df.groupby('batsman')['batsman'].count().sort_values(ascending=False).head(1).index[0]
```

Out[81]:
```
'MS Dhoni'
```

find V Kohli's record against all teams

In [82]:
```
temp_df = ipl[ipl['batsman'] == 'V Kohli']

temp_df.groupby('bowling_team')['batsman_runs'].sum().reset_index()
```

Vishal Acharya

Out[82]:

	bowling_team	batsman_runs
0	Chennai Super Kings	749
1	Deccan Chargers	306
2	Delhi Capitals	66
3	Delhi Daredevils	763
4	Gujarat Lions	283
5	Kings XI Punjab	636
6	Kochi Tuskers Kerala	50
7	Kolkata Knight Riders	675
8	Mumbai Indians	628
9	Pune Warriors	128
10	Rajasthan Royals	370
11	Rising Pune Supergiant	83
12	Rising Pune Supergiants	188
13	Sunrisers Hyderabad	509

Create a function that can return the highest score of any batsman

In [85]:
```
temp_df = ipl.groupby('batsman').get_group('V Kohli')
temp_df.groupby('match_id')['batsman_runs'].sum().sort_values(ascending=False).head(1).values[0]
```

Out[85]: 113

In [86]:
```
def highest(batsman):
  temp_df = ipl[ipl['batsman'] == batsman]
  return temp_df.groupby('match_id')['batsman_runs'].sum().sort_values(ascending=False).head(1).values[0]
highest('DA Warner')
```

Vishal Acharya

Out[86]: 126

TASK-3

In [1]:
```
import pandas as pd
import numpy as np
```

In [4]:
```
fifa_df=pd.read_csv("D:\python_2 _2025_26\Pandas\PANDAS_DATA\Fifa Worldcup 2022.csv")
```

Perform groupby based on the "Team" column and then perform Z Normalization on top of the below columns of each group:

Passes

Passes Completed

Attempted Line Breaks

Completed Line Breaks

You have to make a python function named z_normalization which takes two arguments:

group: Every group that you have created

cols_to_perform: This parameter takes a list of columns on which you have to perform the Z-Normalization.

For this problem, you have to use th apply() method.

Z − Normalization=Xi−μstd

After that find out the below values for each group:

minimum "Passess"

maximum "Passess"

Vishal Acharya

minimum "Yellow Cards"

maximum "Yellow cards"

average "Yellow Cards"

maximum "Attempted Line Breaks"

minimum "Attempted Line Breaks"

standardard deviation of "Attempted Line Breaks"

average Possession

In [8]:
```
groups = fifa_df.groupby("Team", group_keys=False)

groups = groups.apply(z_normalization, cols_to_perform=cols_to_perform)

groups = groups.groupby("Team")

groups.agg({
    "Passes": ["min", "max"],
    "Yellow Cards": ["min", "max", "mean"],
    "Attempted Line Breaks": ["max", "min", "std"],
    "Possession (%)": "mean"
})
```

```
C:\Users\VISHAL\AppData\Local\Temp\ipykernel_20408\3391369538.py:3: FutureWarning: DataFrameGroupBy.apply operated on the group
ing columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the oper
ation. Either pass `include_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to s
ilence this warning.
  groups = groups.apply(z_normalization, cols_to_perform=cols_to_perform)
```

Vishal Acharya

Out[8]:

	Passes		Yellow Cards			Attempted Line Breaks			Possession (%)
	min	max	min	max	mean	max	min	std	mean
Team									
Argentina	408	862	0	8	2.285714	249	141	38.774685	49.285714
Australia	286	466	0	3	1.750000	171	133	17.682383	31.250000
Belgium	512	685	1	3	1.666667	195	167	14.422205	49.000000
Brazil	548	695	0	3	1.200000	193	164	10.691118	50.400000
Cameroon	295	500	1	5	2.666667	182	144	19.857828	38.333333
Canada	448	536	2	4	2.666667	176	102	37.753587	44.333333
Costa Rica	231	454	1	3	2.000000	154	86	38.974351	27.333333
Croatia	461	724	0	2	1.142857	259	97	54.499891	47.428571
Denmark	537	650	1	2	1.666667	241	173	34.000000	51.333333
Ecuador	429	484	0	2	1.000000	177	143	17.009801	42.666667
England	508	810	0	1	0.200000	208	133	29.676590	57.200000
France	358	734	0	3	1.142857	179	126	21.407609	44.857143
Germany	352	827	0	3	1.000000	256	143	56.624494	53.000000
Ghana	319	460	2	4	2.666667	188	122	33.020196	37.000000
Iran	232	463	2	3	2.333333	195	56	77.526340	31.666667
Japan	225	591	0	3	1.500000	253	104	64.048810	30.250000
Korea Republic	359	548	1	2	1.500000	201	139	27.072434	42.250000
Mexico	356	500	1	4	2.333333	198	127	40.706265	46.666667
Morocco	229	583	0	3	1.285714	157	128	10.641339	34.285714
Netherlands	403	786	0	4	1.800000	198	168	12.759310	44.000000

Vishal Acharya

	Passes		Yellow Cards			Attempted Line Breaks			Possession (%)
	min	max	min	max	mean	max	min	std	mean
Team									
Poland	303	466	1	3	1.750000	156	79	36.326070	32.000000
Portugal	461	678	0	3	1.200000	197	165	13.049904	53.800000
Qatar	424	457	0	4	2.333333	174	159	7.767453	39.333333
Saudi Arabia	267	560	2	6	4.666667	205	147	29.022979	36.666667
Senegal	277	511	1	3	1.750000	184	143	16.938615	38.500000
Serbia	403	513	2	7	4.000000	184	140	22.188586	42.666667
Spain	647	1070	0	1	0.500000	237	186	24.289916	67.000000
Switzerland	403	512	1	4	2.250000	178	157	9.032349	43.000000
Tunisia	328	501	1	3	1.666667	200	153	24.419937	38.666667
United States	427	572	0	4	1.250000	204	143	27.693260	47.000000
Uruguay	390	528	1	3	2.000000	186	146	22.030282	41.000000
Wales	325	508	1	2	1.666667	208	104	53.028294	41.000000

TASK-ball by ball dataset

In [11]:
```
balls = pd.read_csv("PANDAS_DATA\ipl_deliveries.csv")
```

Find batsman in below category-

Highest score while chasing

Best Strike rate while chasing and have faced 100+ balls

Chasing mean team batting in second inning

Vishal Acharya

In [12]:
```
# code here
#1
df = balls[balls.innings == 2]
df.groupby(["ID", "batter"]).sum().sort_values(by = "batsman_run", ascending = False)["batsman_run"].head()
```

Out[12]:
```
ID       batter
501206   PC Valthaty      120
501243   V Sehwag         119
1254061  SV Samson        119
1136620  SR Watson        117
336018   ST Jayasuriya    114
Name: batsman_run, dtype: int64
```

In [14]:
```
#2
temp_df = df[~(df.extra_type == "wides")]
temp_df = temp_df.groupby("batter").agg(
{
    "batsman_run": "sum",
    "ballnumber": "count"
})
temp_df["strike_rate"] = temp_df["batsman_run"] / temp_df["ballnumber"] * 100
temp_df[temp_df["ballnumber"] >= 100].sort_values("strike_rate", ascending = False).reset_index().head()
```

Out[14]:

	batter	batsman_run	ballnumber	strike_rate
0	PJ Cummins	222	114	194.736842
1	AD Russell	986	570	172.982456
2	LS Livingstone	182	107	170.093458
3	SP Narine	599	356	168.258427
4	SO Hetmyer	330	200	165.000000

Most Successful bowler against any batsman. Find that pair of bowler and batsman.

Most Successful in terms of dissmissal. A bowler who have dissmissed any batsman most no of times. If any two pairs have same no of dissmisal, consider runs conceded by bowler to that batsman. Those who have concede lesser runs is more successful.

Vishal Acharya

In [15]:
```
# code here
balls["IsBatterOut"] = balls.batter == balls.player_out
balls.groupby(["bowler", "batter"]).agg({
    "IsBatterOut": "sum",
    "batsman_run": "sum"
}).sort_values(by=["IsBatterOut", "batsman_run"], ascending = [False, True]).head()
```

Out[15]:

bowler	batter	IsBatterOut	batsman_run
Z Khan	MS Dhoni	7	74
Sandeep Sharma	V Kohli	7	78
A Mishra	RG Sharma	7	87
R Ashwin	RV Uthappa	7	123
SP Narine	RG Sharma	7	137

Most successful batting pair in IPL. Batting pair who have scored most runs playing together.

In [16]:
```
# code here
def func(x):
    return '-'.join(list(np.sort(x.values)))
balls["batter-pair"] = balls[["batter", "non-striker"]].apply(func, axis = 1)
balls.groupby("batter-pair")["total_run"].sum().sort_values(ascending = False).head()
```

Out[16]:
```
batter-pair
AB de Villiers-V Kohli    3134
CH Gayle-V Kohli          2802
DA Warner-S Dhawan        2357
G Gambhir-RV Uthappa      1906
KL Rahul-MA Agarwal       1731
Name: total_run, dtype: int64
```

Make a dataframe for all batting pairs played together.

Vishal Acharya

Batsman1 Batsman2 Runs Avg StrikeRate

Just to ease this question you can count wide-balls for strike rate.

In [18]:
```
# code here
temp_df = balls.groupby("batter-pair").agg(
{
    "total_run": "sum",
    "ballnumber": "count",
    "isWicketDelivery": "sum"
}).reset_index()
temp_df["Batsman 1"] = temp_df["batter-pair"].apply(lambda x: x.split("-")[0])
temp_df["Batsman 2"] = temp_df["batter-pair"].apply(lambda x: x.split("-")[1])
temp_df.rename(columns = {"total_run": "Runs"}, inplace=True)
temp_df["StrikeRate"] = temp_df["Runs"] / temp_df["ballnumber"] * 100
temp_df["Avg"] = temp_df["Runs"] / temp_df["isWicketDelivery"]
temp_df.sort_values("Runs", ascending = False, inplace=  True)
temp_df[["Batsman 1", "Batsman 2", "Runs", "Avg", "StrikeRate"]]
```

Vishal Acharya

Out[18]:

	Batsman 1	Batsman 2	Runs	Avg	StrikeRate
302	AB de Villiers	V Kohli	3134	44.140845	152.209811
1251	CH Gayle	V Kohli	2802	52.867925	142.017233
1508	DA Warner	S Dhawan	2357	48.102041	136.637681
1954	G Gambhir	RV Uthappa	1906	39.708333	133.754386
2803	KL Rahul	MA Agarwal	1731	52.454545	142.939719
...	...	...	...	...	...
350	AD Mascarenhas	Gurkeerat Singh	0	0.000000	0.000000
3552	NLTC Perera	RV Gomez	0	0.000000	0.000000
2270	Iqbal Abdulla	TS Mills	0	0.000000	0.000000
3556	NM Coulter	Nile	0	0.000000	0.000000
2928	Kartik Tyagi	Shashank Singh	0	0.000000	0.000000

4259 rows × 5 columns

In [ ]: